LATIHAN TERSTRUKTUR

TUGAS PERTEMUAN 4: IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS

Git & Github Dasar



Nama : Risma Marisa

NIM : 3224019

Prodi : Sistem Informasi

Matkul : Implementasi Pengujian Perangkat Lunak

Tanggal : 12 Oktober 2026

BAGIAN – A GIT & GITHUB DASAR

SOAL 1 . Pemahaman Dasar Git dan GitHub

---

1. **Perbedaan Centrailzed VCS dan Distributed VCS beserta Contohnya**
    1. **Centralized Version Control System (CVCS)**

        Centralized Version Control System (CVCS) merupakan sistem pengelolaan versi yang menyimpan file proyek dan riwayat perubahannya pada satu server pusat. Pengguna mengambil file dari server tersebut, mengerjakan perubahan, kemudian mengirimkan hasilnya kembali ke server agar dapat diakses oleh anggota tim lainnya. Sistem ini memudahkan pengelolaan proyek dari satu tempat, tetapi pengguna cukup bergantung pada ketersediaan server untuk melakukan operasi yang membutuhkan akses ke repository pusat.

        Contoh CVCS adalah sebagai berikut.

        1. **Subversion (SVN)**

            Subversion atau SVN merupakan sistem pengelolaan versi yang menggunakan repository pusat untuk menyimpan file proyek dan mencatat perubahan yang dilakukan oleh pengguna. Misalnya, dalam pengembangan aplikasi kampus, beberapa programmer dapat mengambil kode dari server SVN, memperbaikinya, kemudian mengirimkan perubahan tersebut kembali ke server. Dengan begitu, anggota tim dapat menggunakan satu repository pusat sebagai sumber kode proyek yang dikelola bersama.

        2. **Concurrent Versions System (CVS)**

            CVS merupakan sistem pengelolaan versi yang memungkinkan beberapa pengguna bekerja pada file proyek yang sama dengan menyimpan riwayat perubahan di repository pusat. Contohnya, sebuah tim pengembang website menggunakan CVS untuk mengelola file HTML, CSS, dan program lainnya. Ketika seorang anggota memperbaiki kode, perubahan tersebut dapat dikirimkan ke repository pusat sehingga anggota lain dapat mengambil versi terbaru. CVS termasuk sistem yang lebih lama dan memiliki keterbatasan dibandingkan beberapa sistem pengelolaan versi modern.

    2. **Distributed Version Control System (DVCS)**

        Distributed Version Control System (DVCS) adalah sistem pengelolaan versi yang

memungkinkan setiap pengguna memiliki salinan repository lengkap, termasuk riwayat perubahan proyek. Dengan sistem ini, pengguna dapat melakukan banyak pekerjaan secara lokal, seperti melihat riwayat perubahan dan membuat commit, tanpa harus selalu terhubung ke server pusat. Jika pekerjaan sudah siap dibagikan, pengguna dapat mengirimkan perubahan ke repository bersama agar dapat diakses oleh anggota tim lainnya.

Contoh DVCS adalah sebagai berikut.

1. **Git**

   Git merupakan sistem pengelolaan versi terdistribusi yang digunakan untuk mencatat perubahan file dan membantu pengembangan proyek secara individu maupun kelompok. Contohnya, mahasiswa membuat aplikasi menggunakan Git di komputer pribadi. Setiap kali menyelesaikan fitur, mahasiswa tersebut dapat membuat commit untuk menyimpan catatan perubahan secara lokal. Setelah itu, hasil pekerjaannya dapat dikirimkan ke GitHub menggunakan perintah git push agar anggota kelompok dapat melihat dan melanjutkan pengembangan aplikasi.

2. **Mercurial**

   Mercurial merupakan sistem pengelolaan versi terdistribusi yang memiliki fungsi serupa dengan Git, yaitu menyimpan riwayat perubahan dan mendukung kerja sama dalam pengembangan perangkat lunak. Misalnya, tim pengembang aplikasi membagi pekerjaan menjadi beberapa bagian. Setiap anggota dapat mengerjakan perubahan pada repository lokal masing-masing, membuat commit, lalu menggabungkan perubahan ketika diperlukan. Dengan cara ini, pekerjaan anggota tim dapat dikelola tanpa harus selalu melakukan setiap aktivitas langsung pada satu server pusat.

3. **Tabel Perbandingan CVCS dan DVCS**

Aspek	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Penyimpanan repository	Repository utama dan riwayat perubahan dikelola pada server pusat.	Setiap pengguna memiliki salinan repository lengkap beserta riwayat perubahannya.
Cara kerja	Pengguna mengambil file dari server pusat dan mengirimkan perubahan kembali ke server tersebut.	Pengguna dapat bekerja pada repository lokal, kemudian berbagi perubahan melalui repository bersama.

Ketergantungan pada server	Lebih bergantung pada ketersediaan server untuk melakukan operasi yang memerlukan akses ke repository pusat.	Lebih fleksibel karena banyak aktivitas, seperti membuat commit dan melihat riwayat, dapat dilakukan secara lokal.
Penggunaan tanpa internet	Pekerjaan yang membutuhkan komunikasi dengan server akan terhambat ketika server tidak dapat diakses.	Banyak pekerjaan lokal tetap dapat dilakukan tanpa internet, tetapi sinkronisasi dengan repository remote memerlukan koneksi.
Riwayat perubahan	Dikelola terutama melalui repository pusat.	Tersimpan dalam repository masing-masing pengguna.
Kerja sama tim	Anggota tim berbagi perubahan melalui server pusat.	Anggota tim dapat bekerja secara lokal dan kemudian menggabungkan atau membagikan perubahan.
Contoh sistem	Subversion (SVN) dan Concurrent Versions System (CVS).	Git dan Mercurial.

Kesimpulan: Perbedaan utama CVCS dan DVCS terletak pada pengelolaan repository dan ketergantungan terhadap server pusat. CVCS memusatkan penyimpanan pada satu server, sedangkan DVCS memberikan salinan repository kepada setiap pengguna. Oleh karena itu, DVCS seperti Git lebih fleksibel untuk pengembangan perangkat lunak karena pengguna dapat mengelola riwayat perubahan secara lokal dan membagikan hasil pekerjaannya ketika diperlukan.

2. Lima Git Commands yang Wajib Dikuasai Beserta Fungsinya

Git commands adalah perintah yang digunakan untuk mengelola perubahan file dalam proyek perangkat lunak. Perintah-perintah ini membantu pengguna memulai repository, menentukan perubahan yang akan disimpan, mencatat riwayat pekerjaan, serta bertukar perubahan dengan repository yang berada di server seperti GitHub.

1. **Git unit**

   Perintah git init digunakan untuk membuat repository Git baru pada sebuah folder proyek. Setelah perintah ini dijalankan, Git membuat direktori konfigurasi internal bernama .git yang digunakan untuk menyimpan informasi repository dan mengelola riwayat perubahan. Perintah ini biasanya digunakan ketika pengguna mulai mengerjakan proyek baru yang belum menggunakan Git.

   Contoh penggunaan:

   mkdir aplikasi-kampus

   cd aplikasi-kampus

   git init

   Perintah pertama membuat folder aplikasi-kampus, perintah kedua masuk ke folder tersebut, dan git init mengaktifkan pengelolaan versi Git di dalamnya. Setelah itu, file yang dibuat dalam folder tersebut dapat mulai dikelola menggunakan perintah Git lainnya. Namun, git init belum otomatis mengirimkan proyek ke GitHub karena repository yang dibuat masih berada di komputer lokal.

2. **Git add**

   Perintah git add digunakan untuk memasukkan file atau perubahan tertentu ke dalam *staging area*. Staging area merupakan tempat untuk menyiapkan perubahan yang akan dimasukkan ke commit berikutnya. Dengan adanya tahap ini, pengguna dapat memilih perubahan mana yang ingin dicatat tanpa harus memasukkan semua file yang berubah sekaligus.

   Contoh penggunaan:

   git add README.md

   Perintah tersebut menyiapkan perubahan pada file README.md agar dapat disertakan dalam commit berikutnya. Jika pengguna ingin menyiapkan seluruh perubahan yang terdeteksi dalam folder proyek, pengguna dapat menjalankan git add .. Setelah perubahan dimasukkan ke staging area, pengguna dapat memeriksanya menggunakan git status sebelum menjalankan git commit.

3. **Git commit**

   Perintah git commit digunakan untuk menyimpan sekumpulan perubahan yang sudah disiapkan dalam staging area ke dalam riwayat repository lokal. Setiap commit

memiliki identitas dan pesan yang menjelaskan perubahan yang dilakukan. Pesan commit membantu pengguna memahami perkembangan proyek dan mengetahui pekerjaan yang sudah diselesaikan pada setiap tahap.

Contoh penggunaan:

git commit -m "Menambahkan README"

Perintah tersebut membuat catatan perubahan dengan pesan Menambahkan README. Perintah ini akan menyimpan perubahan yang sudah disiapkan sebelumnya menggunakan git add. Jika file belum dimasukkan ke staging area, perubahan pada file tersebut belum otomatis disertakan dalam commit. Commit juga belum otomatis mengirimkan perubahan ke GitHub karena penyimpanannya masih berada di repository lokal.

4. **Git push**

   Perintah git push digunakan untuk mengirimkan commit dari repository lokal ke repository remote, misalnya repository di GitHub. Perintah ini diperlukan agar perubahan yang sudah dicatat di komputer pengguna dapat dibagikan kepada anggota tim atau disimpan pada repository online. Sebelum melakukan push, pengguna perlu memastikan repository remote sudah dikonfigurasi dan branch tujuan yang digunakan sudah sesuai.

   Contoh penggunaan:

   git push origin main

   Pada perintah tersebut, origin merupakan nama remote yang biasanya menunjuk ke repository GitHub, sedangkan main adalah branch yang akan dikirimkan. Misalnya, setelah mahasiswa membuat commit berisi perbaikan kode aplikasi di komputer lokal, perintah ini digunakan untuk mengirimkan commit tersebut ke branch main di GitHub. Dengan demikian, perubahan yang sudah dikirimkan dapat dilihat oleh pengguna lain yang memiliki akses ke repository tersebut.

5. **Git pull**

   Perintah git pull digunakan untuk mengambil perubahan terbaru dari repository remote dan menggabungkannya ke branch lokal yang sedang digunakan. Perintah ini bermanfaat ketika anggota tim lain telah mengirimkan perubahan ke GitHub, sehingga pengguna perlu memperbarui repository lokal sebelum melanjutkan pekerjaannya.

Dengan memperbarui repository secara berkala, perbedaan antara versi lokal dan versi remote dapat dikelola dengan lebih baik.

Contoh penggunaan:

git pull origin main

Perintah tersebut mengambil perubahan dari branch main pada remote bernama origin, kemudian mengintegrasikannya ke branch lokal yang sedang aktif. Contohnya, seorang mahasiswa bekerja dalam kelompok pengembangan aplikasi. Ketika temannya memperbaiki kode dan mengirimkan perubahan ke GitHub, mahasiswa tersebut dapat menjalankan git pull origin main untuk mengambil perubahan terbaru sebelum melanjutkan pekerjaannya. Apabila perubahan yang diambil bertentangan dengan perubahan lokal, Git mungkin meminta pengguna menyelesaikan konflik terlebih dahulu.

6. **Fungsi Lima Git Commands**

Perintah	Kegunaan utama	Hasil yang diperoleh
`git init`	Memulai pengelolaan versi pada folder proyek.	Folder proyek menjadi repository Git lokal.
`git add`	Menyiapkan file atau perubahan untuk dicatat.	Perubahan terpilih masuk ke staging area.
`git commit`	Mencatat perubahan yang sudah disiapkan.	Riwayat commit baru tersimpan di repository lokal.
`git push`	Mengirimkan commit lokal ke repository remote.	Perubahan yang dikirim tersedia di repository remote jika proses berhasil.
`git pull`	Mengambil dan mengintegrasikan perubahan dari remote.	Branch lokal diperbarui dengan perubahan yang berhasil diambil.

3. Fungsi .gitignore dalam Repository Git

`.gitignore` merupakan file konfigurasi yang digunakan untuk memberi tahu Git file atau folder mana yang perlu diabaikan agar tidak ikut dilacak sebagai perubahan baru. File ini berguna untuk menghindari masuknya file yang tidak dibutuhkan ke repository, seperti hasil kompilasi, cache, file sementara, atau konfigurasi lokal pada komputer pengembang.

Contoh isi file .gitignore:

```
target/
.idea/
.vscode/
*.class
*.tmp
__pycache__/
.venv/
.env
```

Penjelasan beberapa pola tersebut adalah sebagai berikut.

1. `target/` digunakan untuk mengabaikan folder hasil build pada proyek yang menggunakan Maven.
2. `.idea/` dan `.vscode/` digunakan untuk mengabaikan pengaturan lokal dari editor atau IDE.
3. `*.class` digunakan untuk mengabaikan file hasil kompilasi Java.
4. `*.tmp` digunakan untuk mengabaikan file sementara yang memiliki ekstensi `.tmp`.
5. `__pycache__/` digunakan untuk mengabaikan folder cache Python.
6. `.venv/` digunakan untuk mengabaikan folder lingkungan virtual Python.
7. `.env` digunakan untuk mengabaikan file konfigurasi yang mungkin berisi informasi sensitif, seperti token atau kata sandi.

SOAL 2. Praktik Penggunaan Git Commands

Pada soal ini, praktik dilakukan dengan membuat repository di GitHub, menghubungkannya dengan komputer lokal, mencatat perubahan menggunakan commit, membuat branch baru, kemudian menggabungkan branch tersebut ke branch utama. Langkah-langkah ini bertujuan untuk memahami alur dasar pengelolaan proyek menggunakan Git dan GitHub.

1. Membuat Repository di GitHub

Pertama, buka situs GitHub di https://github.com, lalu masuk menggunakan akun masing-masing. Setelah berhasil masuk, pilih tombol **New repository** untuk membuat repository baru. Isi pengaturan repository sebagai berikut:

- Buka situs GitHub melalui browser.

- Login menggunakan akun GitHub yang dimiliki.
- Setelah masuk ke halaman utama, klik tombol **New repository** atau buka menu pembuatan repository.
- Pada bagian **Repository name**, masukkan nama `latihan-git-[NIM]`.
- Pilih jenis repository **Public** atau **Private** sesuai instruksi dosen.
- Jika tersedia pilihan untuk menambahkan README secara otomatis, jangan centang pilihan tersebut agar repository dibuat dalam keadaan kosong.
- Klik **Create repository** untuk menyelesaikan proses pembuatan.
- Sebagai contoh, jika NIM adalah `3224019`, nama repository menjadi `latihan-git-3224019`. Setelah semua pengaturan sesuai, klik **Create repository**.
- **Hasil yang diharapkan:** Repository berhasil dibuat di GitHub dan memiliki alamat URL yang nantinya digunakan untuk menghubungkan repository online dengan komputer lokal.

2. **Mengkloning Repository ke Komputer Lokal**

   Setelah repository berhasil dibuat, langkah berikutnya adalah melakukan *clone*. Clone merupakan proses menyalin repository dari GitHub ke komputer lokal, termasuk informasi Git yang diperlukan untuk mengelola perubahan.

   Cara melakukannya:

   - Buka repository yang sudah dibuat di GitHub.
   - Klik tombol **Code**, kemudian salin URL HTTPS repository.
   - Buka Git Bash, terminal komputer, atau terminal pada Visual Studio Code.
   - Arahkan terminal ke folder tempat repository akan disimpan.
   - Jalankan perintah berikut dengan mengganti URL sesuai repository milik sendiri.
   - Setelah proses clone selesai, masuk ke folder repository menggunakan perintah:
   - Kemudian, periksa kondisi repository dengan perintah:
   - Perintah `git clone` digunakan untuk menyalin repository dari GitHub, `cd` digunakan untuk masuk ke folder repository, sedangkan `git status` digunakan untuk memeriksa keadaan repository dan branch yang sedang aktif.

3. **Mengatur Identitas Pengguna Git**

Sebelum membuat commit, identitas pengguna Git perlu diperiksa karena nama dan email pengguna akan dicatat pada riwayat commit. Pengaturan ini membantu menunjukkan siapa yang membuat perubahan pada repository

Jalankan perintah berikut di terminal:

git config --global user.name "Nama Lengkap"

git config --global user.email "email@example.com"

Untuk memeriksa apakah pengaturan berhasil, jalankan:

git config --global --list

Perintah tersebut menampilkan konfigurasi Git yang berlaku secara global pada komputer, termasuk nama dan email pengguna yang sudah diatur.

4. Membuat File README.md

Setelah repository siap digunakan, langkah selanjutnya adalah membuat file `README.md`. File ini biasanya digunakan untuk memberikan informasi mengenai proyek, termasuk identitas pembuat dan gambaran singkat isi repository.

Cara melakukannya:

- Pastikan terminal sudah berada di dalam folder repository `latihan-git-[NIM]`.
- Buka folder tersebut menggunakan editor teks, misalnya Visual Studio Code atau Notepad.
- Buat file baru dengan nama `README.md`.
- Masukkan isi file seperti berikut.
- `# Latihan Git dan GitHub`
- `Nama: Nama Lengkap`
- `NIM: NIM Kamu`
- Ganti `Nama Lengkap` dan `NIM Kamu` dengan identitas yang sebenarnya.
- Simpan file di dalam folder repository dengan nama tepat `README.md`.
- Setelah disimpan, periksa kembali nama file dan lokasinya. Pastikan file tidak tersimpan di folder lain atau memiliki ekstensi tambahan seperti `README.md.txt`.

5. Menambahkan README.md ke Staging Area

File yang baru dibuat belum otomatis masuk ke dalam riwayat Git. Oleh karena itu, file tersebut perlu dimasukkan terlebih dahulu ke *staging area*, yaitu tempat untuk menyiapkan perubahan yang akan disimpan melalui commit.

Jalankan perintah:

git add README.md

Setelah itu, periksa kondisi repository:

Perintah git add README.md digunakan untuk menyiapkan file README agar dapat dimasukkan ke commit berikutnya. Sementara itu, git status membantu memastikan file sudah berada pada tahap yang benar.

6. **Membuat Commit Pertama**

   Setelah file README masuk staging area, perubahan tersebut perlu disimpan ke riwayat repository lokal menggunakan perintah commit.

   Jalankan perintah berikut:

   git commit -m "docs: add README"

   Pesan commit harus ditulis tepat seperti instruksi tugas, yaitu docs: add README. Pesan tersebut menjelaskan bahwa perubahan yang dicatat berkaitan dengan penambahan dokumentasi README.

   Untuk memeriksa apakah commit berhasil dibuat, jalankan:

   Perintah ini menampilkan riwayat commit secara ringkas. Jika berhasil, akan terlihat satu entri commit dengan pesan docs: add README.

7. **Mengirimkan Commit README ke GitHub**

   Setelah commit berhasil dibuat, perubahan perlu dikirimkan ke repository GitHub agar file README dapat dilihat secara online.

   Jalankan perintah berikut:

   git push origin main

   Pada perintah tersebut, origin merupakan nama remote yang biasanya mengarah ke repository GitHub, sedangkan main merupakan branch tujuan pengiriman perubahan.

8. **Membuat Branch Baru Bernama feature-test**

   Setelah README dikirimkan, langkah berikutnya adalah membuat branch baru bernama `feature-test`. Branch merupakan jalur pengembangan terpisah yang memungkinkan perubahan dikerjakan tanpa langsung menambahkannya ke branch utama.

   Jalankan perintah:

   git branch feature-test

   Perintah tersebut membuat branch baru dengan nama feature-test. Untuk memastikan branch

sudah terbentuk, jalankan:

git branch

Git akan menampilkan daftar branch lokal yang tersedia. Tanda bintang (*) menunjukkan branch yang sedang aktif. Pada tahap ini, branch feature-test sudah dibuat, tetapi branch aktif biasanya masih main.

9. Berpindah ke Branch feature-test

Setelah branch dibuat, pengguna perlu berpindah ke branch tersebut sebelum membuat file `test.txt`. Hal ini dilakukan agar perubahan yang dibuat tercatat pada branch yang sesuai dengan instruksi tugas.

Jalankan perintah:

git switch feature-test

Kemudian periksa branch yang sedang aktif:

git branch

Jika berhasil, tanda bintang (*) akan berada di depan feature-test. Artinya, aktivitas pengembangan selanjutnya dilakukan pada branch tersebut.

10. Membuat File test.txt

Setelah berada pada branch `feature-test`, buat file baru dengan nama `test.txt`. File ini digunakan untuk memenuhi instruksi tugas mengenai penambahan file pada branch yang terpisah.

Cara melakukannya:

- Buka folder repository `latihan-git-[NIM]`.
- Buat file baru bernama `test.txt`.
- Masukkan teks berikut ke dalam file.
- `File ini dibuat untuk latihan penggunaan branch pada Git.`
- Simpan file di folder repository yang sama dengan `README.md`.
- Kembali ke terminal dan jalankan:

  `git status`

  Perintah tersebut digunakan untuk memeriksa apakah Git mendeteksi test.txt sebagai file baru. Karena file belum ditambahkan menggunakan git add, file biasanya ditampilkan sebagai file yang belum dilacak (*untracked file*).

11. Menambahkan dan Melakukan Commit pada test.txt

File `test.txt` yang baru dibuat perlu dimasukkan ke staging area sebelum disimpan ke riwayat repository.

Pertama, jalankan:

git add test.txt

Perintah tersebut menyiapkan file test.txt untuk dicatat dalam commit. Setelah itu, jalankan perintah berikut:

git commit -m "test: add test file"

Pastikan pesan test: add test file muncul dalam riwayat commit. Karena commit dilakukan ketika branch aktif adalah feature-test, perubahan tersebut tercatat pada branch tersebut.

12. Mengirimkan Branch feature-test ke GitHub

Setelah file `test.txt` berhasil di-commit, branch `feature-test` perlu dikirimkan ke GitHub agar branch dan perubahannya tersedia di repository online.

Jalankan perintah:

git push -u origin feature-test

Perintah tersebut mengirimkan branch feature-test beserta commit yang ada di dalamnya ke repository remote. Opsi -u digunakan untuk menghubungkan branch lokal dengan branch remote yang sesuai sehingga pengiriman atau pengambilan perubahan berikutnya menjadi lebih mudah.

13. Kembali ke Branch main

Setelah branch `feature-test` dikirimkan, langkah selanjutnya adalah kembali ke branch utama `main`. Hal ini perlu dilakukan karena perubahan dari `feature-test` akan digabungkan ke branch yang sedang aktif.

Jalankan perintah:

git switch main

Kemudian periksa branch aktif menggunakan:

git branch

Pastikan tanda bintang (*) berada di depan main. Jika branch utama di GitHub memiliki perubahan baru yang belum ada di komputer lokal, pengguna dapat mengambil perubahan tersebut terlebih dahulu dengan:

git pull origin main

Perintah git pull origin main digunakan untuk mengambil dan mengintegrasikan perubahan

terbaru dari branch main pada repository remote. Langkah ini berguna apabila repository online sudah berubah sejak terakhir kali dilakukan clone atau push.

14. Mengambungkan feature-test ke main

Setelah berada pada branch `main`, lakukan proses penggabungan atau *merge* untuk memasukkan perubahan dari branch `feature-test` ke branch utama.

Jalankan perintah:

git merge feature-test

Perintah tersebut menggabungkan perubahan dari branch feature-test ke branch yang sedang aktif, yaitu main. Karena file test.txt sebelumnya dibuat pada feature-test, file tersebut akan ikut masuk ke branch main setelah merge berhasil.

Setelah merge berhasil, periksa isi folder repository untuk memastikan test.txt sudah tersedia ketika branch main aktif.

15. Mengirimkan Hasil Merge ke GitHub

Proses merge yang dilakukan di komputer lokal belum otomatis mengubah isi repository di GitHub. Oleh karena itu, hasil penggabungan perlu dikirimkan kembali ke repository remote.

Jalankan perintah:

```
git push origin main
```

Perintah tersebut mengirimkan perubahan pada branch lokal main ke branch main di GitHub. Jika proses berhasil, repository online akan menerima hasil penggabungan yang sebelumnya dilakukan di komputer lokal.

16. Memeriksa Kondisi Akhir Repository

Tahap terakhir adalah memeriksa kembali repository untuk memastikan seluruh proses sudah dilakukan dengan benar. Pemeriksaan ini penting agar tidak ada perubahan yang tertinggal atau langkah yang belum selesai.

Jalankan perintah berikut satu per satu:

```
git status
```

Perintah ini digunakan untuk memeriksa kondisi file dan perubahan yang belum disimpan. Jika semua perubahan lokal sudah dicatat dan tidak ada pekerjaan yang tertinggal, biasanya Git menampilkan pesan bahwa working tree bersih (*working tree clean*).

Selanjutnya, jalankan:

```
git branch
```

Perintah ini menampilkan daftar branch lokal. Pastikan `main` dan `feature-test` tersedia.

Untuk melihat riwayat perubahan, jalankan:

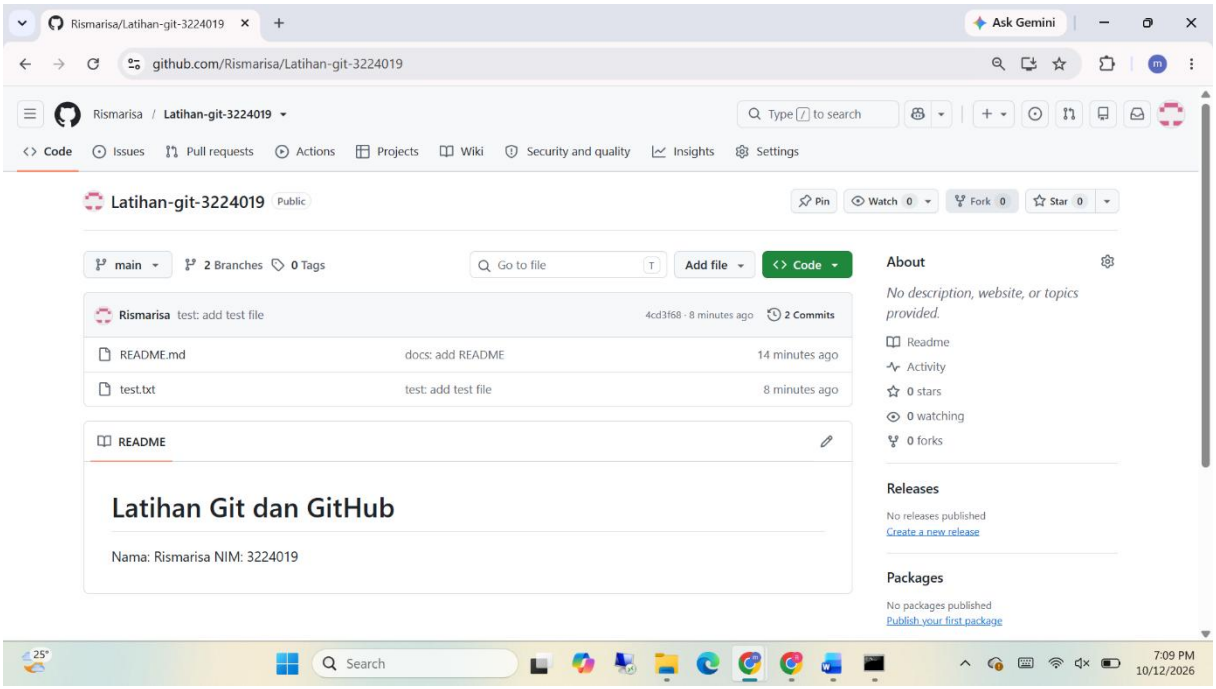
```
git log --oneline
```

Perintah ini menampilkan daftar commit secara ringkas. Periksa apakah pesan `docs: add README` dan `test: add test file` sudah tercatat.

Terakhir, jalankan:

```
git ls-files
```

Perintah tersebut menampilkan file yang sedang dilacak oleh Git. Pastikan `README.md` dan `test.txt` termasuk dalam daftar file.



Berdasarkan hasil praktik yang telah dilakukan, repository Latihan-git-3224019 berhasil dibuat dan dihubungkan dengan GitHub. File README.md berhasil dibuat dengan commit docs: add README, sedangkan file test.txt dibuat pada branch feature-test dengan commit test: add test file. Selanjutnya, branch feature-test berhasil digabungkan ke branch main dan hasilnya berhasil dikirim ke GitHub. Berdasarkan tampilan repository, kedua file sudah tersedia pada branch main dan terdapat dua riwayat commit. Dengan demikian, praktik penggunaan Git, mulai dari pembuatan branch, commit, push, hingga merge, telah berhasil dilakukan.

BAGIAN – B UNIT TESTING FRAMEWORK

SOAL 3. Koversi Tst Casa ke Kode Unit Test

---

1. **Menentukan 5 Test Case Berdasarkan Independent Path**

   A. Analisis Program

   Program validasiPendaftaran() merupakan fungsi yang digunakan untuk memeriksa apakah data yang dimasukkan pengguna sudah memenuhi persyaratan pendaftaran. Proses validasi dilakukan secara berurutan, dimulai dari pemeriksaan nama, umur, email, hingga persetujuan terhadap syarat dan ketentuan.

   Pada setiap kondisi, program dapat mengembalikan pesan kesalahan apabila data tidak memenuhi persyaratan. Jika semua kondisi terpenuhi, program akan menampilkan pesan "Pendaftaran berhasil". Oleh karena itu, unit testing diperlukan untuk memastikan bahwa setiap kondisi menghasilkan keluaran yang sesuai.

   B. Lima Test Case

No.	Kondisi yang Diuji	Input Nama	Umur	Email	Setuju Syarat	Hasil
1.	Nama kosong	""	20	andi@gmail.com	true	Nama harus diisi
2.	Umur di bawah 17 tahun	Andi	20	andi@gmail.com	true	Umur minimal 17 tahun
3.	Email tidak memiliki karakter @	Andi	20	andi@gmail.com	true	Email tidak valid
4.	Belum menyetujui syarat	Andi	20	andi@gmail.com	false	Harus menyetujui syarat dan ketentuan
5.	Semua data memenuhi persyaratan	Andi	20	andi@gmail.com	true	Pendaftaran berhasil

C. Penjelasan Setiap Test Case

- Test case pertama digunakan untuk menguji kondisi ketika nama tidak diisi. Input nama diberikan dalam bentuk string kosong (""), sedangkan data lainnya dibuat valid. Program akan memeriksa kondisi nama == null || nama.isEmpty(). Karena nama kosong, kondisi tersebut bernilai benar sehingga program mengembalikan pesan "Nama harus diisi". Pengujian ini bertujuan memastikan bahwa pengguna tidak dapat melanjutkan pendaftaran tanpa mengisi nama.
- Test case kedua digunakan untuk menguji batas umur minimum. Pada pengujian ini, nama diisi dengan Andi, umur diberikan nilai 16 tahun, email menggunakan andi@gmail.com, dan pengguna menyetujui syarat. Karena nama sudah terisi, program melanjutkan pemeriksaan ke bagian umur. Kondisi umur < 17 bernilai benar karena umur pengguna masih di bawah 17 tahun. Oleh sebab itu, hasil yang diharapkan adalah pesan "Umur minimal 17 tahun". Pengujian ini memastikan bahwa aturan batas umur diterapkan sesuai dengan ketentuan program.
- Test case ketiga digunakan untuk memeriksa validasi email. Nama dan umur sudah memenuhi persyaratan, tetapi email yang digunakan adalah andigmail.com, yaitu alamat yang tidak mengandung karakter @. Ketika program memeriksa kondisi email == null || !email.contains("@"), kondisi tersebut bernilai benar sehingga program mengembalikan pesan "Email tidak valid". Pengujian ini bertujuan memastikan bahwa program dapat menolak email yang tidak memenuhi pemeriksaan sederhana sebagaimana ditetapkan dalam kode.
- Test case keempat digunakan untuk menguji persetujuan terhadap syarat dan ketentuan. Pada kondisi ini, nama, umur, dan email sudah memenuhi persyaratan, tetapi nilai setujuSyarat diberikan false. Program kemudian memeriksa kondisi !setujuSyarat. Karena pengguna belum menyetujui syarat, program mengembalikan pesan "Harus menyetujui syarat dan ketentuan". Pengujian ini memastikan bahwa persetujuan pengguna menjadi salah satu syarat yang harus dipenuhi sebelum pendaftaran dinyatakan berhasil.
- Test case kelima digunakan untuk menguji kondisi ketika seluruh data memenuhi persyaratan. Nama diisi dengan Andi, umur 20 tahun, email menggunakan andi@gmail.com, dan nilai persetujuan diberikan true. Dalam kondisi ini, tidak ada

satu pun pemeriksaan kesalahan yang terpenuhi. Program kemudian mencapai perintah return terakhir dan menghasilkan pesan "Pendaftaran berhasil". Pengujian ini bertujuan memastikan bahwa program dapat menerima pendaftaran ketika seluruh ketentuan telah dipenuhi.

D. Hubungan Test Case dengan Independent Path

Kelima test case digunakan untuk menguji jalur eksekusi yang berbeda pada fungsi validasiPendaftaran(). Empat test case menguji kondisi kesalahan, yaitu nama kosong, umur kurang dari 17 tahun, email tidak valid, dan belum menyetujui syarat. Sementara itu, satu test case lainnya menguji kondisi ketika seluruh persyaratan terpenuhi. Dengan pengujian ini, setiap kondisi dapat diperiksa untuk memastikan program menghasilkan keluaran yang sesuai dengan yang diharapkan.

2. Implementasi Test Case Menggunakan Kode Unit Test Junit

JUnit merupakan framework pengujian pada Java yang digunakan untuk memeriksa apakah suatu program menghasilkan keluaran sesuai dengan yang diharapkan. Dalam soal ini, JUnit digunakan untuk menguji method validasiPendaftaran() berdasarkan lima test case yang telah disusun. Implementasi pengujian dilakukan dengan membuat kode program dan kode pengujian secara terpisah agar proses validasi dapat diperiksa secara sistematis.

A. Kode Program ValidasiPendaftaran.java

```
public class ValidasiPendaftaran {
ublic String validasiPendaftaran(String nama, int umur, String email, boolean
setujuSyarat) {
    if (nama == null || nama.isEmpty()) {
       return "Nama harus diisi";
    }

    if (umur < 17) {
       return "Umur minimal 17 tahun";
    }

    if (email == null || !email.contains("@")) {
       return "Email tidak valid";
```

```
        }

        if (!setujuSyarat) {
            return "Harus menyetujui syarat dan ketentuan";
        }

        return "Pendaftaran berhasil";
    }
}
```

Kode tersebut merupakan implementasi method validasi yang akan diuji. Pemeriksaan dilakukan secara berurutan, dimulai dari nama, umur, email, dan persetujuan syarat. Jika salah satu kondisi tidak terpenuhi, program langsung mengembalikan pesan kesalahan yang sesuai. Apabila seluruh kondisi dilewati tanpa kesalahan, program mengembalikan pesan "Pendaftaran berhasil".

B. Kode Unit Test ValidasiPendaftaranTest.java

```
mport org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;


public class ValidasiPendaftaranTest {


  ValidasiPendaftaran validasi = new ValidasiPendaftaran();


  @Test
  public void testNamaKosong() {
    assertEquals(
      "Nama harus diisi",
      validasi.validasiPendaftaran("", 20, "andi@gmail.com", true)
    );
  }
```

```
@Test
public void testUmurKurangDari17() {
    assertEquals(
        "Umur minimal 17 tahun",
        validasi.validasiPendaftaran("Andi", 16, "andi@gmail.com", true)
    );
}

@Test
public void testEmailTidakValid() {
    assertEquals(
        "Email tidak valid",
        validasi.validasiPendaftaran("Andi", 20, "andigmail.com", true)
    );
}

@Test
public void testBelumSetujuSyarat() {
    assertEquals(
        "Harus menyetujui syarat dan ketentuan",
        validasi.validasiPendaftaran("Andi", 20, "andi@gmail.com", false)
    );
}

@Test
public void testPendaftaranBerhasil() {
    assertEquals(
        "Pendaftaran berhasil",
        validasi.validasiPendaftaran("Andi", 20, "andi@gmail.com", true)
    );
}
```

```
}
```

C. Penjelasan Kode Unit Test

Pada kode pengujian tersebut, anotasi @Test digunakan untuk menandai method yang akan dijalankan oleh JUnit sebagai pengujian. Objek ValidasiPendaftaran dibuat agar setiap test dapat memanggil method yang sedang diperiksa. Sementara itu, assertEquals() digunakan untuk membandingkan hasil yang diharapkan dengan hasil aktual dari program. Jika kedua hasil sama, pengujian tersebut dinyatakan berhasil. Sebaliknya, jika hasilnya berbeda, JUnit akan menunjukkan bahwa pengujian gagal.

Setiap method pengujian mewakili satu test case yang telah disusun sebelumnya. Method testNamaKosong() memeriksa apakah program menampilkan pesan ketika nama tidak diisi. Method testUmurKurangDari17() memeriksa penolakan terhadap umur di bawah batas minimum. Method testEmailTidakValid() menguji email yang tidak mengandung karakter @, sedangkan testBelumSetujuSyarat() memeriksa kondisi ketika pengguna belum menyetujui syarat dan ketentuan. Terakhir, testPendaftaranBerhasil() memastikan bahwa program memberikan pesan keberhasilan ketika seluruh input memenuhi persyaratan. Dengan memisahkan setiap pengujian ke dalam method tersendiri, hasil pengujian menjadi lebih mudah diperiksa dan kesalahan dapat diketahui berdasarkan kondisi yang diuji.

D. Cara Menjalankan Pengujian

Untuk menjalankan pengujian, proyek harus menggunakan JUnit 5 dan memiliki konfigurasi yang sesuai. Jika menggunakan Maven, kode program dapat diletakkan pada folder src/main/java, sedangkan kode pengujian diletakkan pada folder src/test/java, dengan package yang sesuai. Dependency JUnit Jupiter juga perlu ditambahkan pada file pom.xml. Setelah konfigurasi selesai, pengujian dapat dijalankan melalui terminal pada direktori utama proyek dengan perintah berikut:

mvn test

Setelah perintah dijalankan, hasil pengujian dapat diperiksa melalui keluaran terminal. Jika kelima pengujian berhasil, berarti hasil method sesuai dengan keluaran yang diharapkan untuk kelima test case tersebut. Jika terdapat pengujian yang gagal, pesan dari JUnit dapat

digunakan untuk mengetahui perbedaan antara hasil aktual dan hasil yang seharusnya.

SOAL 4. Implementasi Unit Test untuk Kode StudentGrade

1. **Menghitung Cyclomatic Comlexity Method calculateGrade()**

   Program StudentGrade digunakan untuk menentukan nilai huruf dan status kelulusan mahasiswa berdasarkan skor yang diperoleh. Ketentuan nilai yang digunakan dalam program adalah sebagai berikut.

Rentang Skor	Nilai Huruf	Status
80-100	A	Lulus
70-79	B	Lulus
60-69	C	Lulus
50-59	D	Tidak Lulus
0-49	E	Tidak Lulus
Di bawah 0 atau di atas 100	Tidak Valid	Exception

   Berdasarkan ketentuan tersebut, program memiliki dua method utama, yaitu calculateGrade() untuk menentukan nilai huruf dan isPassed() untuk menentukan status kelulusan mahasiswa. Kedua method memiliki validasi agar skor yang dimasukkan hanya berada dalam rentang 0 sampai 100.

2. **Kode Program StudentGrade.java**

   Berikut adalah kode program yang akan dianalisis dan di uji menggunakan Junit.

```
public class StudentGrade {


   public String calculateGrade(int score) {
      if (score < 0 || score > 100) {
         throw new IllegalArgumentException("Score must be between 0 and
100");
      }

      if (score >= 80) return "A";
      else if (score >= 70) return "B";
      else if (score >= 60) return "C";
```

```
        else if (score >= 50) return "D";
        else return "E";
    }

    public boolean isPassed(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException("Score must be between 0 and
100");
        }

        return score >= 60;
    }
}
```

Pada kode tersebut, method calculateGrade() menentukan nilai huruf berdasarkan skor yang dimasukkan. Apabila skor berada di luar rentang 0 sampai 100, program akan menghasilkan IllegalArgumentException. Jika skor valid, program menentukan nilai A, B, C, D, atau E sesuai dengan kondisi yang telah ditetapkan.

Sementara itu, method isPassed() digunakan untuk menentukan kelulusan mahasiswa. Mahasiswa dinyatakan lulus apabila memperoleh skor minimal 60, sehingga method mengembalikan true. Jika skor kurang dari 60, method mengembalikan false.

3. **Menghitung Cyclomatic Complexity**

Cyclomatic Complexity merupakan ukuran yang digunakan untuk mengetahui jumlah jalur independen dalam suatu program berdasarkan struktur percabangannya. Nilai ini dapat membantu menentukan seberapa banyak jalur yang perlu dipertimbangkan saat menyusun pengujian.

Rumus yang digunakan adalah:

V(G) = P + 1

Keterangan:

- **V(G)** adalah nilai Cyclomatic Complexity.
- **P** adalah jumlah titik keputusan atau kondisi percabangan.

A. Cyclomatic Complexity Method calculateGrade()

Pada method calculateGrade(), terdapat lima titik keputusan yang diperhitungkan, yaitu:

- Kondisi validasi skor score < 0 || score > 100.
- Kondisi score >= 80.
- Kondisi score >= 70.
- Kondisi score >= 60.
- Kondisi score >= 50.

Dengan demikian, perhitungannya adalah:

V(G) = P + 1

V(G) = 5 + 1

V(G) = 6

Jadi, nilai Cyclomatic Complexity method calculateGrade() adalah **6**. Artinya, terdapat enam jalur independen yang menjadi dasar dalam penyusunan pengujian, meliputi skor tidak valid dan lima kemungkinan hasil nilai huruf.

B. Cyclomatic Complexity Method isPassed()

Pada method isPassed(), terdapat dua titik keputusan yang diperhitungkan, yaitu kondisi validasi skor score < 0 || score > 100 dan kondisi kelulusan score >= 60.

Perhitungannya adalah:

V(G) = P + 1

V(G) = 2 + 1

V(G) = 3

Jadi, nilai Cyclomatic Complexity method isPassed() adalah **3**. Nilai tersebut menunjukkan bahwa pengujian perlu mencakup jalur skor tidak valid, jalur skor yang memenuhi syarat lulus, dan jalur skor yang tidak memenuhi syarat lulus.

4. Menentukan Independent Path

Independent path merupakan jalur eksekusi yang berbeda dalam suatu program. Setiap jalur ditentukan berdasarkan kondisi yang dilewati hingga program menghasilkan nilai atau melemparkan exception.

A. Independent Path Method calculateGrade()

Berdasarkan hasil perhitungan Cyclomatic Complexity sebesar 6, terdapat enam jalur

independen yang dapat dijelaskan sebagai berikut.

1. Path 1 Skor tidak valid: Skor kurang dari 0 atau lebih dari 100. Program melemparkan IllegalArgumentException.
2. Path 2 Nilai A: Skor valid dan memenuhi kondisi score >= 80. Program mengembalikan nilai "A".
3. Path 3 Nilai B: Skor valid, kondisi score >= 80 tidak terpenuhi, tetapi score >= 70 terpenuhi. Program mengembalikan nilai "B".
4. Path 4 Nilai C: Skor valid, kondisi sebelumnya tidak terpenuhi, tetapi score >= 60 terpenuhi. Program mengembalikan nilai "C".
5. Path 5 Nilai D: Skor valid, kondisi sebelumnya tidak terpenuhi, tetapi score >= 50 terpenuhi. Program mengembalikan nilai "D".
6. Path 6 Nilai E: Skor valid dan seluruh kondisi nilai A, B, C, serta D tidak terpenuhi. Program mengembalikan nilai "E".

B. Independent Path Method isPassed()

Berdasarkan hasil perhitungan Cyclomatic Complexity sebesar 3, terdapat tiga jalur yang perlu diuji.

1. Path 1 Skor tidak valid Skor kurang dari 0 atau lebih dari 100. Program melemparkan IllegalArgumentException.
2. Path 2 Lulus Skor valid dan memenuhi kondisi score >= 60. Program mengembalikan true.
3. Path 3 Tidak lulus Skor valid, tetapi kurang dari 60. Program mengembalikan false.

Independent path tersebut digunakan sebagai dasar untuk menentukan test case. Dengan menguji setiap jalur, kita dapat memeriksa apakah program menghasilkan keluaran yang sesuai dengan kondisi yang telah ditentukan.

5. Implementasi Unit Test Menggunakan Jubit 5

JUnit merupakan framework pengujian pada Java yang digunakan untuk memeriksa apakah suatu method memberikan hasil sesuai dengan yang diharapkan. Pengujian pada program StudentGrade dilakukan terhadap method calculateGrade() dan isPassed(), termasuk pengujian skor yang tidak valid.

A. Kode Unit Test StudentGradeTest.java

Berikut adalah kode pengujian menggunakan JUnit 5.

```
import org.junit.jupiter.api.Test;
```

```
import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

   StudentGrade studentGrade = new StudentGrade();

   @Test
   public void testGradeA() {
      assertEquals("A", studentGrade.calculateGrade(85));
   }

   @Test
   public void testGradeB() {
      assertEquals("B", studentGrade.calculateGrade(75));
   }

   @Test
   public void testGradeC() {
      assertEquals("C", studentGrade.calculateGrade(65));
   }

   @Test
   public void testGradeD() {
      assertEquals("D", studentGrade.calculateGrade(55));
   }

   @Test
   public void testGradeE() {
      assertEquals("E", studentGrade.calculateGrade(40));
   }

   @Test
   public void testScoreDiBawahNol() {
      assertThrows(IllegalArgumentException.class,
            () -> studentGrade.calculateGrade(-1));
   }

   @Test
   public void testScoreDiAtasSeratus() {
      assertThrows(IllegalArgumentException.class,
            () -> studentGrade.calculateGrade(101));
   }

   @Test
   public void testLulus() {
      assertTrue(studentGrade.isPassed(60));
```

```
    }

    @Test
    public void testTidakLulus() {
       assertFalse(studentGrade.isPassed(59));
    }

    @Test
    public void testIsPassedScoreTidakValid() {
       assertThrows(IllegalArgumentException.class,
            () -> studentGrade.isPassed(101));
    }
}
```

B. Tabel Skenario Pengujian

Pengujian dilakukan menggunakan sepuluh test case. Setiap test case memiliki input, hasil yang diharapkan, dan tujuan pengujian yang berbeda.

No.	Method	Input	Hasil yang Diharapkan	Tujuan
1.	calculateGrade()	85	"A"	Menguji nilai A
2.	calculateGrade()	75	"B"	Menguji nilai B
4.	calculateGrade()	55	"D"	Menguji nilai D
5.	calculateGrade()	40	"E"	Menguji nilai E
6.	calculateGrade()	-1	IllegalArgumentException	Menguji skor di bawah 0
7.	calculateGrade()	101	IllegalArgumentException	Menguji skor di atas 100
8.	isPassed()	60	True	Menguji batas minimal kelulusan
9.	isPassed()	59	false	Menguji skor di bawah batas kelulusan
10.	isPassed()	101	IllegalArgumentException	Menguji validasi skor pada method kelulusan

C. PenjelasaN Kode Unit Test

Kode pengujian menggunakan beberapa assertion dari JUnit 5. Anotasi @Test digunakan untuk menandai method yang akan dijalankan sebagai pengujian. Sementara itu,

assertEquals() digunakan untuk membandingkan hasil aktual dengan nilai yang diharapkan, misalnya memastikan bahwa skor 85 menghasilkan nilai A.

Method assertTrue() digunakan untuk memastikan bahwa skor 60 dinyatakan lulus, sedangkan assertFalse() digunakan untuk memastikan bahwa skor 59 dinyatakan tidak lulus. Adapun assertThrows() digunakan untuk memeriksa apakah program menghasilkan IllegalArgumentException ketika menerima skor yang tidak valid.

Melalui sepuluh test case tersebut, pengujian mencakup setiap kategori nilai, batas kelulusan, serta validasi skor. Dengan demikian, kesalahan pada logika penentuan nilai maupun status kelulusan dapat diketahui lebih awal.

6. Cara Menjalankan Pengujian

Untuk menjalankan pengujian, proyek Java perlu dikonfigurasi menggunakan JUnit 5. Jika menggunakan Maven, struktur folder yang digunakan adalah sebagai berikut.

File StudentGrade.java berisi kode program utama, sedangkan StudentGradeTest.java berisi seluruh kode pengujian. Kedua file perlu berada pada package yang sesuai agar class dapat dikenali dengan benar.

Pastikan dependency JUnit Jupiter sudah ditambahkan pada file pom.xml. Setelah itu, buka terminal pada direktori utama proyek dan jalankan perintah berikut.

mvn test

Perintah tersebut akan menjalankan test case yang terdapat pada proyek. Jika semua pengujian berhasil, hasilnya menunjukkan bahwa program memberikan keluaran sesuai dengan yang diharapkan untuk seluruh kasus yang diuji. Apabila ada pengujian yang gagal, periksa pesan kesalahan untuk mengetahui bagian program yang belum sesuai.

BAGIAN – C GITHUB ACTIONS

SOAL 5. Analisis Workflow YAML

---

1. Apa nama workflow di atas?

Workflow pada kode tersebut bernama Python CI. Nama ini digunakan sebagai identitas workflow yang nantinya dapat dilihat pada menu Actions di GitHub. Workflow ini berfokus pada pengujian kode Python secara otomatis agar developer dapat mengetahui apakah program masih berjalan dengan baik setelah terjadi perubahan kode.

2. Kapan workflow tersebut akan dijalankan?

Pengaturan waktu jalannya workflow terdapat pada bagian berikut:.

```
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
```

Berdasarkan konfigurasi tersebut, workflow akan berjalan ketika developer mengirim perubahan kode ke branch main atau develop. Selain itu, workflow juga akan aktif ketika ada pull request yang ditujukan ke branch main.

Pengaturan ini berguna agar kode yang mengalami perubahan dapat diperiksa melalui pengujian otomatis. Dengan begitu, kemungkinan kesalahan dapat diketahui sebelum perubahan kode digunakan lebih lanjut.

3. Fungsi Strategy Matrix pada Workflow

Strategy matrix merupakan fitur GitHub Actions yang memungkinkan satu job dijalankan beberapa kali menggunakan kombinasi konfigurasi yang berbeda. Pada workflow tersebut, matrix digunakan untuk menguji kode Python menggunakan tiga versi Python, yaitu 3.10, 3.11, dan 3.12.

Bagian konfigurasi yang menunjukkan penggunaan matrix adalah:

```
strategy:
  matrix:
    python-version: ['3.10', '3.11', '3.12']
```

Strategy matrix berfungsi untuk mengatur pengujian program menggunakan beberapa versi Python dalam satu konfigurasi workflow. Pada kode tersebut, pengujian dilakukan dengan Python versi 3.10, 3.11, dan 3.12.

Setiap versi akan menjalankan rangkaian pengujian yang sama. Hal ini membantu developer mengetahui apakah program dapat berjalan pada versi Python yang berbeda atau justru mengalami masalah pada versi tertentu.

Sebagai contoh, apabila program berhasil diuji menggunakan Python 3.10 dan 3.11, tetapi gagal pada Python 3.12, developer dapat memeriksa bagian kode atau dependency yang mungkin tidak kompatibel dengan versi tersebut.

Jadi, penggunaan strategy matrix dapat menghemat waktu pengujian dan membantu menjaga

kompatibilitas program tanpa perlu membuat workflow terpisah untuk setiap versi Python.

4. **Sebutkan 5 step yang ada di dalam job test!**

   Pada job test, terdapat lima steps yang dijalankan secara berurutan. Setiap step memiliki fungsi masing-masing dalam menyiapkan lingkungan pengujian, memasang kebutuhan, dan menjalankan test.

   A. Step 1 Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

   Langkah ini bertugas menyediakan kode proyek dari repository GitHub ke lingkungan tempat workflow dijalankan. Kode tersebut diperlukan sebagai bahan yang akan diuji. Setelah source code tersedia, proses dapat dilanjutkan ke tahap persiapan Python.

   B. Step 2 Setup Python

```
- name: Setup Python ${{ matrix.python-version }}
  uses: actions/setup-python@v5
  with:
    python-version: ${{ matrix.python-version }}
```

   Langkah ini mempersiapkan Python sesuai dengan versi yang sedang diproses oleh strategy matrix. Karena terdapat tiga versi Python yang ditentukan, workflow akan menjalankan pengujian pada masing-masing versi tersebut. Tujuannya agar pengujian tidak hanya bergantung pada satu versi Python saja.

   C. Step 3 Install Dependencies

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install pytest pytest-cov
```

   Step ini Pada tahap ini, workflow memasang paket yang dibutuhkan untuk menjalankan pengujian. Perintah pertama memperbarui pip, sedangkan perintah kedua memasang pytest dan pytest-cov.

   pytest digunakan untuk menjalankan unit test, sementara pytest-cov membantu menghitung code coverage. Kedua paket tersebut diperlukan agar pengujian dan pengukuran cakupan kode dapat dilakukan pada tahap berikutnya.

D. Step 4 Run Tests

```
- name: Run tests
  run: pytest --cov=src --cov-report=xml
```

Langkah terakhir menjalankan seluruh test yang ditemukan oleh pytest sekaligus mengukur cakupan kode pada direktori src. Bagian --cov=src menentukan kode yang akan diukur, sedangkan --cov-report=xml menghasilkan laporan coverage dalam format XML.

Hasil pengujian dapat digunakan untuk mengetahui apakah test berhasil dijalankan dan bagian mana dari kode yang sudah tercakup oleh pengujian. Apabila ditemukan test yang gagal, developer dapat memeriksa hasil dan log pengujian untuk mencari penyebabnya.

E. Step 5 Tidak Ada Step Kelima yang Terpisah pada YAML

Berdasarkan YAML yang diberikan, sebenarnya hanya terdapat **empat steps** di dalam job test, bukan lima. Keempatnya adalah Checkout code, Setup Python, Install dependencies, dan Run tests.

Baris python -m pip install --upgrade pip dan pip install pytest pytest-cov merupakan dua perintah dalam satu step bernama Install dependencies, bukan dua steps terpisah. Oleh karena itu, jika soal meminta lima steps, terdapat ketidaksesuaian antara jumlah yang diminta dengan YAML yang ditampilkan.

Ringkasan step yang benar-benar ada dalam kode adalah sebagai berikut.

No.	Nama Step	Fungsi
1.	Checkout Code	Mengambilkan kode repository runner.
2.	Setup Python	Menyiapkan versi Python sesuai matrix
3.	Install dependencies	Memperbarui pip dan memasang pytest serta pytest-cov.
4.	Run tests	Menjalankan pengujian dan menghasilkan laporan coverage XML.

BAGIAN – D CODE COVERAGE & INTEGRASI

SOAL 6. Membuat Workflow Java Unit Test

Pada soal ini, dibuat workflow GitHub Actions untuk proyek Java yang berfungsi menjalankan unit testing secara otomatis. Workflow menggunakan runner ubuntu-latest dan JDK 17 sebagai lingkungan untuk menjalankan program. Pengujian dilakukan menggunakan Maven melalui perintah mvn test, kemudian dilanjutkan dengan pembuatan laporan *code coverage* menggunakan JaCoCo. Setelah laporan dibuat, hasilnya akan diunggah sebagai artifact dengan nama jacoco-report.

File Workflow YAML

File workflow disimpan pada lokasi:

.**github/workflows/java-unit-test.yml**

Isi lengkap file YAML adalah sebagai berikut:

```
name: Java Unit Test

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
```

```
      java-version: '17'
      distribution: 'temurin'

    - name: Run tests
      run: mvn test

    - name: Generate coverage report
      run: mvn jacoco:report

    - name: Upload coverage report
      uses: actions/upload-artifact@v4
      with:
        name: jacoco-report
        path: target/site/jacoco/
        if-no-files-found: error
```

1. **Nama Workflow**

```
name: Java Unit Test
```

Bagian ini digunakan untuk menentukan nama workflow yang akan ditampilkan pada menu Actions di GitHub. Nama Java Unit Test menunjukkan bahwa workflow tersebut digunakan untuk menjalankan pengujian unit pada proyek Java.

2. **Trigger atau Event**

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
```

Bagian on berfungsi menentukan kapan workflow dijalankan. Pada konfigurasi tersebut, terdapat dua jenis pemicu, yaitu:

- push, yang menjalankan workflow ketika perubahan kode dikirim ke branch main atau develop.
- pull_request, yang menjalankan workflow ketika terdapat pull request yang ditujukan ke branch main.

  Pengaturan ini memungkinkan pengujian dilakukan secara otomatis ketika terdapat perubahan kode pada branch yang telah ditentukan.

3. Job dan Runner

```
jobs:
  test:
    runs-on: ubuntu-latest
```

Bagian jobs digunakan untuk mendefinisikan pekerjaan yang akan dilakukan oleh GitHub Actions. Pada workflow ini, job diberi nama test. Sementara itu, runs-on: ubuntu-latest menentukan bahwa proses pengujian akan dijalankan pada lingkungan Ubuntu yang disediakan oleh GitHub Actions.

4. Step 1 Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

Step pertama menggunakan actions/checkout@v4 untuk mengambil kode sumber dari repository GitHub ke dalam runner. Proses ini diperlukan agar file proyek tersedia dan dapat digunakan selama pengujian. File yang diambil mencakup kode program, unit test, dan konfigurasi Maven.

5. Step 2 Set up JDK 17

```
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    java-version: '17'
```

```
distribution: 'temurin'
```

Step kedua digunakan untuk menyiapkan lingkungan Java dengan JDK versi 17. Action actions/setup-java@v4 membantu memasang dan mengatur versi Java yang akan digunakan oleh runner. Penggunaan JDK 17 memastikan proses kompilasi dan pengujian berjalan menggunakan versi Java yang sesuai dengan konfigurasi proyek.

6. Step 3 Run Tests

```
- name: Run tests

  run: mvn test
```

Step ketiga menjalankan unit test menggunakan perintah mvn test. Perintah tersebut digunakan oleh Maven untuk menjalankan proses kompilasi dan pengujian yang tersedia di proyek. Hasilnya dapat menunjukkan jumlah test yang dijalankan, test yang berhasil, serta test yang mengalami kegagalan atau error.

7. Step 4 Generate Coveraage Report

```
- name: Generate coverage report

  run: mvn jacoco:report
```

Step keempat digunakan untuk menghasilkan laporan *code coverage* dengan JaCoCo. Laporan ini menunjukkan bagian kode program yang telah dijalankan selama proses unit testing. Secara default, hasil laporan disimpan di direktori target/site/jacoco/, dengan syarat konfigurasi JaCoCo pada pom.xml sudah sesuai dan data pengujian tersedia.

8. Step 5 Upload Coverage Report

```
- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: target/site/jacoco/
    if-no-files-found: error
```

Step kelima digunakan untuk mengunggah hasil laporan JaCoCo sebagai artifact GitHub Actions. Artifact tersebut diberi nama jacoco-report, sedangkan file yang diunggah berasal dari direktori target/site/jacoco/. Pengaturan if-no-files-found: error digunakan agar proses upload dinyatakan gagal apabila file laporan tidak ditemukan.

Setelah workflow berhasil dijalankan, artifact dapat dilihat dan diunduh melalui halaman Actions pada repository GitHub. Dengan demikian, hasil laporan *code coverage* dapat diperiksa kembali tanpa harus menjalankan pengujian dari awal.

9. Hasil Implementasi Workflow

Setelah file YAML dimasukkan ke repository dan perubahan dikirim ke GitHub, workflow akan berjalan sesuai dengan trigger yang telah ditentukan. Hasil yang diharapkan adalah seluruh lima step utama berhasil dijalankan, mulai dari pengambilan kode sumber, penyiapan JDK, pelaksanaan unit test, pembuatan laporan JaCoCo, hingga pengunggahan artifact jacoco-report.

Keberhasilan implementasi perlu dibuktikan dengan menjalankan workflow pada repository GitHub. Jika terdapat kesalahan konfigurasi, unit test gagal, atau laporan tidak terbentuk, step terkait perlu diperiksa dan diperbaiki terlebih dahulu.

SOAL 7. Analisis Code Coverage

kode program yang sudah dijalankan ketika unit test dilakukan. Berdasarkan hasil coverage report yang diberikan pada soal, terdapat empat file Python yang diuji, yaitu calculator.py, login_system.py, payment.py, dan utils.py.

1. **Hitung total statement yang ada di project**

   Berdasarkan kolom **Stmts** pada coverage report, jumlah statement dari setiap file adalah sebagai berikut:

   - src/calculator.py = 10 statement
   - src/login_system.py = 15 statement
   - src/payment.py = 20 statement
   - src/utils.py = 5 statement
   - Perhitungan total statement:
   - 10 + 15 + 20 + 5 = **50 statement**

     Jadi, total statement yang ada dalam project tersebut adalah **50 statement**. Artinya, terdapat 50 statement yang menjadi bagian dari perhitungan code coverage pada project.

2. **Hitung total statement yang tidak tereksekusi (Miss)!**

   Kolom **Miss** menunjukkan jumlah statement yang belum dijalankan ketika unit test dilakukan. Berdasarkan laporan, jumlah statement yang tidak tereksekusi pada setiap file adalah:

   - src/calculator.py = 0 statement
   - src/login_system.py = 2 statement
   - src/payment.py = 8 statement
   - src/utils.py = 1 statement
   - Perhitungan total statement yang tidak tereksekusi:
   - 0 + 2 + 8 + 1 = **11 statement**

     Jadi, total statement yang tidak tereksekusi adalah **11 statement**. Hal ini menunjukkan bahwa masih ada 11 statement yang belum tercakup oleh pengujian. Oleh karena itu, test case perlu diperiksa dan dilengkapi agar lebih banyak bagian kode dapat dijalankan saat pengujian.

3. **File mana yang memiliki coverage paling rendah? Berapa persen?**

   Berdasarkan hasil coverage rwport, persentase coberage setiap file adalah:

   - src/calculator.py = 10 statement
   - src/login_system.py = 15 statement

- src/payment.py = 20 statement
- src/utils.py = 5 statement
- File yang memiliki coverage paling rendah adalah **`src/payment.py`, dengan persentase coverage sebesar 60%**

  Persentase tersebut menunjukkan bahwa cakupan pengujian pada file payment.py masih lebih rendah dibandingkan file lainnya. Selain itu, file ini memiliki 8 statement yang tidak tereksekusi, yaitu jumlah Miss paling banyak dalam laporan. Karena itu, file payment.py perlu menjadi prioritas untuk diperiksa dan ditambahkan test case-nya.

4. Apa yang harus dilakukan untuk meningkatkan coverage pada file payment.py?

Untuk meningkatkan coverage pada file payment.py, ada beberapa langkah yang dapat dilakukan:

1. Memeriksa kode program. Pelajari fungsi dan kondisi yang terdapat di dalam file payment.py untuk mengetahui bagian mana yang belum dijalankan saat pengujian.
2. Menambahkan test case. Buat pengujian tambahan untuk bagian kode yang belum tercakup. Pengujian perlu disesuaikan dengan fungsi yang ada pada file tersebut, termasuk kondisi normal maupun kondisi yang dapat menimbulkan kesalahan.
3. seperti if-else, setiap kondisi yang relevan perlu diuji. Misalnya, apabila kode memiliki validasi jumlah pembayaran, pengujian dapat mencakup jumlah pembayaran yang valid, tidak valid, atau kondisi batas sesuai aturan program.
4. Menjalankan pengujian kembali. Setelah test case ditambahkan, jalankan kembali unit test dan buat laporan coverage. Hasilnya kemudian diperiksa untuk mengetahui apakah statement yang sebelumnya belum tereksekusi sudah tercakup.
5. Dengan langkah-langkah tersebut, coverage pada file payment.py diharapkan meningkat dari 60%. Namun, peningkatan persentase harus diikuti dengan pengujian yang benar-benar memeriksa hasil program, bukan hanya menjalankan kode tanpa memastikan kebenaran hasilnya.

5. Menurut Anda, apakah coverage 78% sudah cukup baik? Jelaskan alasannya!

Menurut saya, coverage 78% sudah cukup baik sebagai tahap awal, tetapi masih perlu ditingkatkan. Berdasarkan laporan, dari total 50 statement, sebanyak 39 statement sudah tercakup oleh pengujian, sedangkan 11 statement belum tereksekusi. Artinya, sebagian besar kode sudah diuji, tetapi masih ada bagian yang belum diperiksa melalui unit test.

Coverage sebesar 78% juga belum tentu menjamin bahwa program bebas dari kesalahan. Code coverage hanya menunjukkan seberapa banyak bagian kode yang dijalankan ketika pengujian berlangsung, bukan secara langsung membuktikan bahwa seluruh hasil program sudah benar. Oleh karena itu, test case tetap harus dibuat dengan tepat dan mencakup berbagai kondisi yang mungkin terjadi.

BAGIAN – E STUDI KASUS TERPADU

SOAL 8.

---

1. Implementasi Project

Pada studi kasus ini, saya membuat project Java berbasis Maven dengan nama BankAccountProject. Project ini digunakan untuk menerapkan unit testing pada class BankAccount yang memiliki tiga method utama, yaitu deposit(double amount) untuk menambahkan saldo, withdraw(double amount) untuk mengurangi saldo, dan getBalance() untuk mengetahui saldo rekening saat ini.

Pada class BankAccount, saya menambahkan validasi agar nominal transaksi tidak boleh negatif. Selain itu, proses penarikan saldo tidak diperbolehkan apabila jumlah yang ditarik melebihi saldo yang tersedia. Validasi ini bertujuan agar saldo rekening tetap sesuai dengan aturan yang sudah ditentukan.

Project ini menggunakan Java 17 dan Maven. Pengujian dilakukan menggunakan JUnit 5, sedangkan JaCoCo digunakan untuk mengukur code coverage. Selanjutnya, GitHub Actions digunakan untuk menjalankan pengujian dan membuat laporan coverage secara otomatis ketika terdapat perubahan pada repository.

2. Tools dan Teknologi yang Digunakan

Project ini menggunakan beberapa tools dan teknologi, yaitu:

Tools/Teknologi	Fungsi
Java 17	Bahasa dan environment utama project.
Maven	Mengelola project, dependency, dan proses build.
Junit 5	Framework untuk menjalankan unit testing.

JaCoCo	Mengukur code coverage.
Git	Version control untuk mencatat perubahan kode.
GitHub	Repository untuk menyimpan project secara online.
GitHub Actions	Menjalankan proses Continuous Integration secara otomatis.

3. Unit Testing

Sesuai ketentuan soal yang meminta minimal 6 test case, pada project ini saya membuat 9 unit test agar pengujian tidak hanya mencakup kondisi normal, tetapi juga nilai batas dan kondisi error.

Test yang dilakukan meliputi:

1. Memastikan saldo awal bernilai 0.
2. Memastikan saldo awal bernilai 0.
3. Deposit dengan nominal valid.
4. Deposit lebih dari satu kali untuk memastikan saldo terakumulasi.
5. Deposit dengan nilai 0.
6. Deposit dengan nilai negatif dan memastikan exception muncul.
7. Withdraw dengan nominal valid.
8. Withdraw seluruh saldo.
9. Withdraw dengan nominal lebih besar dari saldo dan memastikan exception muncul.
10. Withdraw dengan nilai negatif dan memastikan exception muncul.

Unit test dijalankan menggunakan perintah:

mvn test

Setelah pengujian dijalankan, hasilnya diperiksa melalui ringkasan yang ditampilkan oleh Maven.

Keterangan	Hasil
Total test case	Diisi dari hasil pengujian

Test yang berhasil	Diisi dari hasil pengujian
Test yang gagal	Diisi dari hasil pengujian
Error	Diisi dari hasil pengujian
Status build	Diisi dari hasil pengujian

Apabila seluruh pengujian berhasil, hasilnya akan menunjukkan 9 test dijalankan tanpa failure maupun error dan status build BUILD SUCCESS. Hasil tersebut perlu dipastikan melalui proses pengujian pada project yang digunakan.

4. **Code Coverage**

   Setelah unit test berhasil dijalankan, code coverage diukur menggunakan JaCoCo. Coverage report dihasilkan melalui perintah:

   mvn jacoco:report

   Laporan JaCoCo digunakan untuk mengetahui bagian kode program yang telah tercakup oleh unit test. Hasil laporan dapat dilihat melalui file target/site/jacoco/index.html.

   Hasil coverage dicatat berdasarkan laporan JaCoCo yang dihasilkan.

Jenis Coverage	Hasil
Instruction Coverage	Diisi sesuai laporan JaCoCo
Branch Coverage	Diisi sesuai laporan JaCoCo
Line Coverage	Diisi sesuai laporan JaCoCo
Method Coverage	Diisi sesuai laporan JaCoCo
Class Coverage	Diisi sesuai laporan JaCoCo

   Apabila laporan menunjukkan coverage 100% pada seluruh metrik tersebut, berarti seluruh bagian yang dianalisis oleh JaCoCo telah tercakup oleh pengujian. Namun, nilai coverage yang tinggi tidak secara otomatis menjamin bahwa program bebas dari kesalahan.

5. Implementasi GitHub Actions

   Setelah unit test dan coverage dijalankan secara lokal, project diintegrasikan dengan GitHub Actions. Workflow dibuat untuk menjalankan proses berikut secara otomatis:

   1. **Checkout code, untuk mengambil source code dari repository.**
   2. **Setup JDK 17, untuk menyiapkan environment Java.**
   3. **Run tests, untuk menjalankan unit test menggunakan Maven.**
   4. **Generate coverage report, untuk menghasilkan laporan JaCoCo.**
   5. **Upload coverage report, untuk mengunggah hasil coverage sebagai artifact.**

Workflow dijalankan ketika terjadi push ke branch main atau develop, serta ketika terdapat pull request ke branch main.

File workflow disimpan pada direktori .github/workflows/ dengan nama bankaccount-ci.yml. Konfigurasi perintah Maven dan lokasi laporan perlu disesuaikan dengan struktur folder project di repository.

6. Hasil Pipeline GitHub Actions

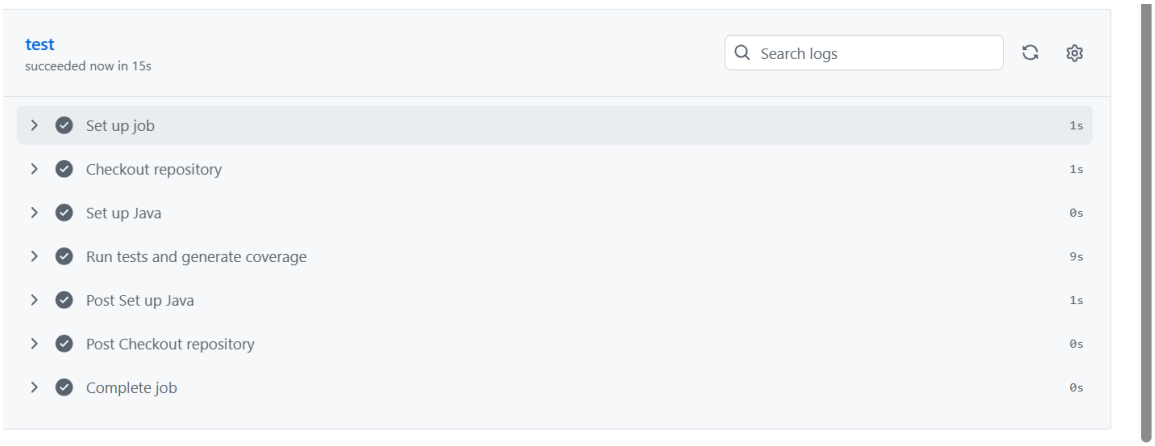
Setelah project diunggah ke repository GitHub, workflow dapat dijalankan melalui push ke branch yang telah dikonfigurasi.

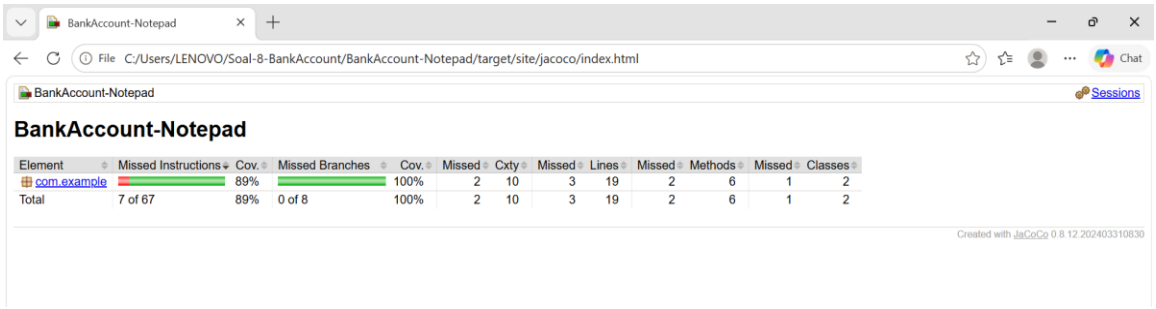
Hasil pipeline diperiksa melalui tab Actions pada repository GitHub. Bagian yang perlu diperiksa meliputi status job test dan keberhasilan setiap step.

Proses	**Hasil**
Checkout code	Diisi sesuai hasil pipeline
Setup JDK 17	Diisi sesuai hasil pipeline
Run tests	Diisi sesuai hasil pipeline
Generate coverage report	Diisi sesuai hasil pipeline
Upload coverage report	Diisi sesuai hasil pipeline

Jika seluruh proses berhasil, GitHub Actions akan menunjukkan status sukses dan menghasilkan artifact bernama jacoco-report. Artifact tersebut dapat digunakan untuk melihat atau mengunduh laporan coverage.

Bukti yang dilampirkan:







7. Repository GitHub

Seluruh source code, unit test, konfigurasi Maven, dan workflow GitHub Actions disimpan pada repository GitHub.

Repository:

Masukkan link repository GitHub milik sendiri setelah project berhasil diunggah.

Struktur utama repository terdiri atas:

```
bank-account-ci/
├── .github/
│   └── workflows/
│       └── bankaccount-ci.yml
├── src/
│   ├── main/java/com/stmik/
│   │   └── BankAccount.java
│   └── test/java/com/stmik/
│       └── BankAccountTest.java
└── pom.xml
```

Kesimpulan

Project BankAccount dirancang untuk menerapkan unit testing menggunakan Java 17 dan Maven, dengan JUnit 5 sebagai framework pengujian dan JaCoCo untuk mengukur code coverage.

Sebanyak 9 test case disiapkan untuk menguji kondisi normal, kondisi batas, dan kondisi error sehingga telah melebihi ketentuan minimal 6 test case.

Project juga diintegrasikan dengan GitHub Actions untuk menjalankan pengujian dan membuat laporan coverage secara otomatis. Keberhasilan implementasi ditentukan berdasarkan hasil pengujian, laporan JaCoCo, dan status pipeline yang benar-benar diperoleh. Seluruh kode program dan konfigurasi teknis disimpan pada repository GitHub, sedangkan screenshot hasil pipeline dan coverage digunakan sebagai bukti implementasi.

Nama	Link Repository
Soal 2	https://github.com/Rismarisa/Latihan-git-3224019.git
Soal 8	https://github.com/Rismarisa/BankAccount-Notepad.git